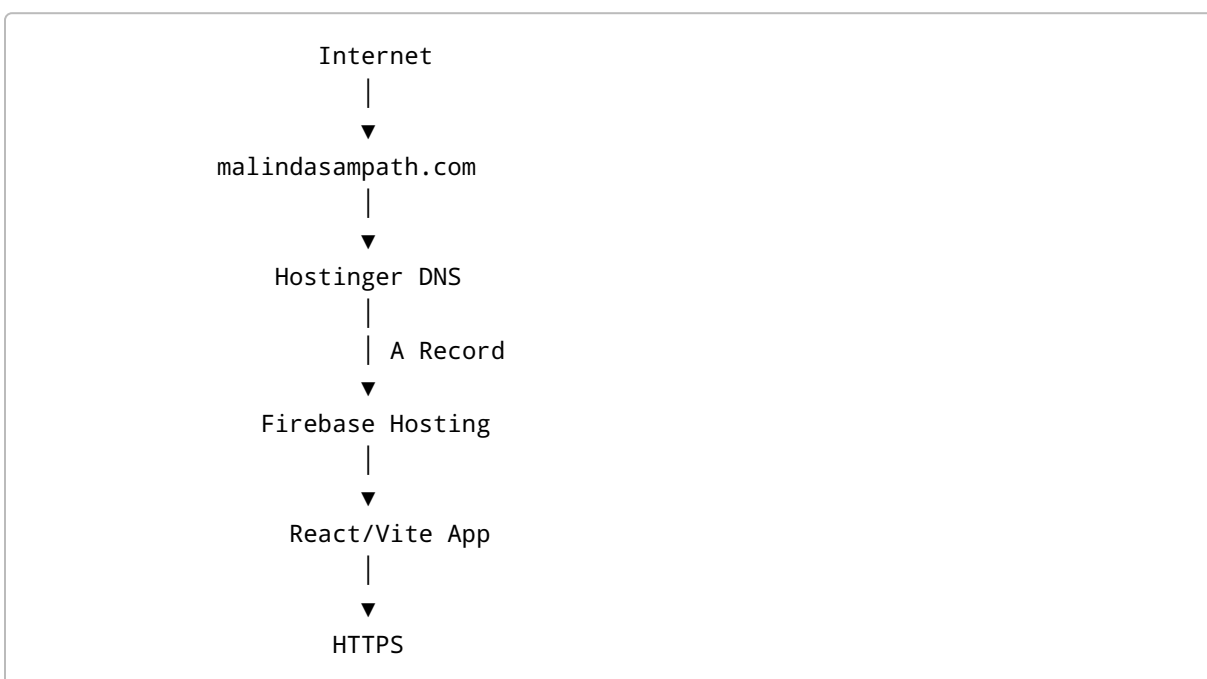


Deploying a React/Vite Portfolio to Firebase Hosting with a Custom Hostinger Domain

1. Introduction

This guide explains how to deploy a **React + Vite portfolio website** to **Firebase Hosting** and connect it to a custom domain purchased from **Hostinger**.

By the end, the architecture will look like this:



The Firebase-provided URL will also remain available:

```
https://YOUR-PROJECT-ID.web.app
```

2. What We Are Building

Our application is a static React/Vite website.

The development project looks approximately like this:

```
portfolio/  
|  
├─ src/  
├─ public/  
├─ package.json  
├─ vite.config.ts  
├─ Dockerfile  
├─ index.html  
└─ ...
```

After building the application:

```
npm run build
```

Vite creates:

```
dist/  
|  
├─ index.html  
├─ assets/  
├─ images/  
├─ icons/  
└─ ...
```

Firebase Hosting serves the contents of this `dist` directory.

Therefore:

```
Source Code  
  |  
  └─ npm run build  
    ▼  
  dist/  
    |  
    └─ Firebase Hosting  
      ▼  
  Website
```

3. Why Firebase Hosting?

For a portfolio website, Firebase Hosting is convenient because the website is essentially a collection of static files.

We don't need:

- A Linux server
- Nginx configuration
- Docker on the production server
- Kubernetes
- A backend server
- A VM

Instead:

```
React/Vite
  ↓
Build
  ↓
dist/
  ↓
Firebase Hosting
```

Firebase also provides HTTPS/SSL for the website.

4. Important Concepts

Before starting, understand these four things.

4.1 React

React is the frontend framework used to build the website.

4.2 Vite

Vite is the build tool used by the React project.

It converts the development source code into optimized production files.

For example:

```
npm run build
```

creates:

```
dist/
```

4.3 Firebase Hosting

Firebase Hosting is where the generated static website files are hosted.

Firebase provides a URL such as:

```
https://YOUR-PROJECT-ID.web.app
```

4.4 DNS

DNS connects your domain name to your hosting provider.

For example:

```
malindasampath.com  
↓  
Firebase Hosting
```

Without DNS, the browser doesn't know where your domain should go.

5. Requirements

You need:

1. A Google account
2. A Google Cloud/Firebase project
3. A Firebase Hosting site
4. A React/Vite project
5. Node.js and npm
6. A domain
7. Access to your domain's DNS settings

In this example:

```
Domain:  
malindasampath.com
```

The domain is managed through:

```
Hostinger
```

6. Step 1 — Make Sure the React Project Works

Open your project locally.

For example:

```
cd /path/to/portfolio
```

Install dependencies:

```
npm install
```

Then start the development server:

```
npm run dev
```

Open the URL shown by Vite, usually:

```
http://localhost:5173
```

Make sure the website works correctly.

7. Step 2 — Build the React Application

Before deploying, create the production build.

Run:

```
npm run build
```

A successful build should look similar to:

```
vite building for production...
```

- ✓ modules transformed.
- ✓ built successfully

A `dist` directory should then appear:

```
portfolio/  
|  
├─ dist/  
|   ├─ index.html  
|   ├─ assets/  
|   └─ images/  
|   ...  
├─ src/  
├─ public/  
├─ package.json  
└─ vite.config.ts
```

The important directory is:

```
dist/
```

This is what Firebase will deploy.

8. Step 3 — Test the Production Build with Docker

This step is optional.

If your project already has a Dockerfile, you can test the production build locally.

Example Dockerfile:

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Build the image:

```
docker build -t portfolio .
```

Run it:

```
docker run -p 8080:80 portfolio
```

Then open:

```
http://localhost:8080
```

If the website works, your production build is good.

Important

The Dockerfile is **not required for Firebase Hosting**.

Firebase will use:

```
dist/
```

directly.

9. Step 4 — Create or Select a Firebase Project

Open the Firebase Console:

<https://console.firebase.google.com/>

Create a Firebase project or add Firebase to an existing Google Cloud project.

In our example, the Firebase project ID is:

```
project-c7396ba2-e46c-4c-17d7a
```

You can find the project ID in the Firebase Console or Google Cloud Console.

10. Step 5 — Register the Web App

Inside Firebase:

```
Project
  ↓
Project Settings
  ↓
Your apps
  ↓
Web application
```

Register your web application.

Firebase will provide a configuration similar to:

```
const firebaseConfig = {
  apiKey: "...",
  authDomain: "...",
  projectId: "...",
  storageBucket: "...",
  messagingSenderId: "...",
  appId: "..."
};
```


Important

For Firebase Hosting, you do **not necessarily need to add the Firebase SDK to your React application**.

The Firebase SDK is required when your application uses Firebase services such as:

- Authentication
- Firestore
- Storage
- Analytics
- Realtime Database

For a simple static portfolio, Firebase Hosting alone is sufficient.

11. Step 6 — Install/Check Firebase CLI

Firebase CLI allows you to deploy from the terminal.

Check whether it is installed:

```
firebase --version
```

Example:

```
15.24.0
```

If it isn't installed, install it using npm:

```
npm install -g firebase-tools
```

Then verify:

```
firebase --version
```

12. Step 7 — Login to Firebase

Run:

```
firebase login
```

On a normal computer, Firebase may open a browser automatically.

For Cloud Shell or other environments where the browser cannot open automatically, use:

```
firebase login --no-localhost
```

Firebase will display a URL.

Open that URL in a browser and authenticate with your Google account.

After successful authentication, you should see something like:

```
Success! Logged in as your-email@gmail.com
```

13. Step 8 — Connect the Project to Firebase

Normally you can run:

```
firebase init hosting
```

Firebase will ask which project to use.

Select:

```
Use an existing project
```

Then select your Firebase project.

If Firebase cannot list projects

Sometimes the Firebase CLI may fail with:

```
Failed to list Firebase projects
```

even though the project exists and your Google account has access.

In that situation, you can manually create `.firebaserc`.

Inside your project:

```
cat > .firebaserc <<'EOF'
{
  "projects": {
    "default": "YOUR-FIREBASE-PROJECT-ID"
  }
}
EOF
```

For this example:

```
{
  "projects": {
    "default": "project-c7396ba2-e46c-4c-17d7a"
  }
}
```

14. Step 9 — Create firebase.json

Create:

```
firebase.json
```

with:

```
{
  "hosting": {
    "public": "dist",
    "ignore": [
      "firebase.json",
      "**/.*",
      "**/node_modules/**"
    ],
    "rewrites": [
      {
        "source": "**",
        "destination": "/index.html"
      }
    ]
  }
}
```

```
}  
]  
}  
}
```

The important part is:

```
"public": "dist"
```

This tells Firebase:

Deploy the contents of the `dist` directory.

15. Why Do We Need the Rewrite?

React applications commonly use client-side routing.

For example:

```
/  
about  
projects  
contact
```

The server needs to send `index.html` to React so React Router can handle the URL.

This configuration:

```
"rewrites": [  
  {  
    "source": "**",  
    "destination": "/index.html"  
  }  
]
```

helps Firebase serve the React application correctly.

16. Step 10 — Deploy the Website

First build the project:

```
npm run build
```

Then deploy:

```
firebase deploy --only hosting
```

If Firebase project selection causes problems, explicitly specify the project:

```
firebase deploy --only hosting --project YOUR-FIREBASE-PROJECT-ID
```

For this example:

```
firebase deploy --only hosting --project project-c7396ba2-e46c-4c-17d7a
```

A successful deployment will show something similar to:

```
✓ Deploy complete!
```

Hosting URL:

```
https://project-c7396ba2-e46c-4c-17d7a.web.app
```

17. Step 11 — Test the Firebase Website

Open the Firebase URL:

```
https://project-c7396ba2-e46c-4c-17d7a.web.app
```

If your portfolio appears, Firebase Hosting is working.

At this point:

```
React/Vite
  ↓
npm run build
  ↓
dist/
  ↓
Firebase Hosting
  ↓
web.app
```

is complete.

18. Step 12 — Add Your Custom Domain

Now we connect:

```
malindasampath.com
```

to Firebase.

Go to:

```
Firebase Console
  ↓
Hosting
  ↓
Custom domains
  ↓
Add custom domain
```

Enter:

```
malindasampath.com
```

Firebase will provide DNS records.

In this example, Firebase provided:

```
A
malindasampath.com
199.36.158.100
```

and:

```
TXT
malindasampath.com
hosting-site=project-c7396ba2-e46c-4c-17d7a
```

19. Step 13 — Configure Hostinger DNS

Open Hostinger:

```
Hostinger
  ↓
Domains
  ↓
malindasampath.com
  ↓
DNS / Manage DNS
```

The old A record was:

```
A
@
4.144.26.113
```

This was pointing the domain to the previous hosting/server.

It must be removed.

20. Add Firebase A Record

Create:

Type: A
Name: @
Content: 199.36.158.100
TTL: 300

The result is:

A @ 199.36.158.100

21. Add Firebase TXT Record

Create:

Type: TXT
Name: @
Content: hosting-site=project-c7396ba2-e46c-4c-17d7a
TTL: 300

The result is:

TXT @ hosting-site=project-c7396ba2-e46c-4c-17d7a

This TXT record proves to Firebase that you control the domain.

22. Configure www

For:

www.malindasampath.com

we used:

Type: CNAME
Name: www


```
Content: malindasampath.com
TTL: 300
```

Therefore:

```
www.malindasampath.com
  ↓
malindasampath.com
  ↓
Firebase Hosting
```

Firebase can then handle the domain.

23. Final DNS Configuration

The final Hostinger configuration should look approximately like:

Type	Name	Content
A	@	199.36.158.100
TXT	@	hosting-site=project-c7396ba2-e46c-4c-17d7a
CNAME	www	malindasampath.com

Do not delete unrelated DNS records.

For example, if you have email records:

```
MX
TXT
```

keep them unless you know they are no longer required.

24. Verify DNS

You can verify DNS from Cloud Shell or any terminal.

Check the A record:

```
nslookup -type=A malindasampath.com
```

Expected:

```
Address: 199.36.158.100
```

Check the TXT record:

```
nslookup -type=TXT malindasampath.com
```

Expected:

```
hosting-site=project-c7396ba2-e46c-4c-17d7a
```

If both records appear, DNS is configured correctly.

25. Firebase SSL Certificate

After DNS is configured, Firebase will verify the domain and create an SSL certificate.

Firebase may display:

```
Minting certificate.  
This may take up to 24 hours.
```

This is normal.

The process is:

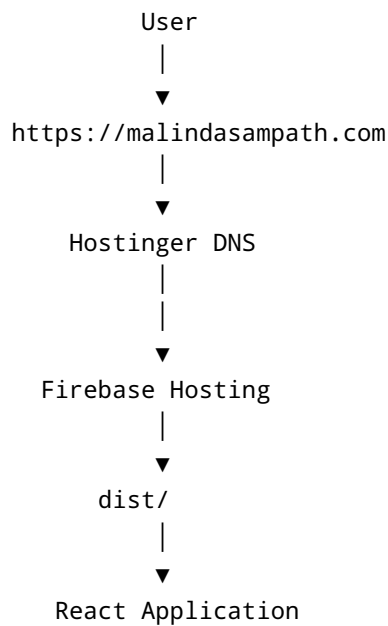
```
DNS configured  
  ↓  
Firebase verifies domain  
  ↓  
Firebase creates SSL certificate  
  ↓  
HTTPS enabled
```

↓
Custom domain becomes active

You don't need to manually create an SSL certificate.

26. Final Website Architecture

After everything is complete:



The Firebase URL also remains available:

`https://project-c7396ba2-e46c-4c-17d7a.web.app`

27. Updating the Website Later

One of the best things about this setup is that updating the portfolio is simple.

After making changes to the React project:

```
git pull
```

or make your code changes manually.

Then:

```
npm install
```

if dependencies changed.

Build:

```
npm run build
```

Deploy:

```
firebase deploy --only hosting --project project-c7396ba2-e46c-4c-17d7a
```

That's it.

The same domain:

```
https://malindasampath.com
```

will automatically serve the new version.

28. Recommended Deployment Workflow

For normal development:

```
1. Modify React code
  ↓
2. Test locally
  ↓
3. npm run build
  ↓
4. Verify dist/
  ↓
5. firebase deploy
```

↓
6. Website updated

For example:

```
npm run build
firebase deploy --only hosting --project project-c7396ba2-e46c-4c-17d7a
```

29. GitHub Integration

Your source code can remain on GitHub.

For example:

```
GitHub
  |
  | source code
  ▼
React/Vite project
  |
  | npm run build
  ▼
dist/
  |
  | firebase deploy
  ▼
Firebase Hosting
```

GitHub is therefore your **source-code repository**, while Firebase is your **hosting platform**.

They have different responsibilities.

30. Firebase vs Docker

Your project also has a Dockerfile.

You can use either approach depending on the hosting platform.

Firestore Hosting

```
React
↓
Vite build
↓
dist
↓
Firestore Hosting
```

Docker + Cloud Run

```
React
↓
Vite build
↓
Docker image
↓
Nginx
↓
Cloud Run
```

For a simple static portfolio, Firestore Hosting is simpler.

You don't need:

```
Docker
Nginx
VM
Kubernetes
```

for this deployment.

31. Firestore Hosting vs Google Cloud VM

A portfolio does not normally need a VM.

With a VM:

```
VM
↓
```

```
Linux
↓
Nginx
↓
SSL
↓
Firewall
↓
Docker
↓
Application
```

You would have to manage these components.

With Firebase:

```
Firebase Hosting
↓
Upload dist/
↓
Done
```

Firebase handles much of the infrastructure.

32. Is It Free?

Firebase Hosting has a no-cost tier, subject to Google's current usage limits.

For a normal personal portfolio with low traffic, the free tier is generally sufficient.

However, **"free" does not mean unlimited.**

Usage can be affected by:

- Traffic
- Data transfer
- Storage
- Other Firebase/Google Cloud services

A portfolio with normal personal traffic is usually a very small workload.

Also remember that your **domain registration itself is not free** if you purchased the domain from Hostinger.

Your costs are therefore conceptually:

```
Domain registration
      +
Firebase Hosting
      ↓
Firebase can be $0 within the no-cost limits
```

33. Important: Large Images

Before publishing a portfolio, optimize images.

For example, if an image is:

```
28 MB
```

that's extremely large for a website.

Large images cause:

- Slow page loading
- Higher bandwidth usage
- Poor mobile performance
- Higher hosting usage

Prefer optimized formats such as:

```
WebP
AVIF
```

and resize images to the actual dimensions needed on the website.

For example:

```
Original:
4000 × 4000
28 MB

Better:
```



```
1200 × 1200  
200–500 KB
```

The exact size depends on the image.

34. Common Problems

Problem 1 — Firebase cannot list projects

Error:

```
Failed to list Firebase projects
```

Try:

```
firebase login --reauth
```

or:

```
firebase login --no-localhost
```

You can also explicitly specify the project:

```
firebase deploy --only hosting --project YOUR-PROJECT-ID
```

Problem 2 — Firebase says project doesn't exist

Check the project:

```
gcloud projects describe YOUR-PROJECT-ID
```

Make sure you are using the **Firebase project ID**, not another project ID.

Problem 3 — Website works on web.app but not custom domain

Check DNS:

```
nslookup -type=A yourdomain.com
```

The result should match the Firebase-provided A record.

Problem 4 — Firebase says TXT record not detected

Check:

```
nslookup -type=TXT yourdomain.com
```

You should see the Firebase verification value.

For example:

```
hosting-site=YOUR-PROJECT-ID
```

Problem 5 — Firebase says ACME challenge failed

This usually means Firebase's SSL verification is still reaching an old DNS destination.

Check:

```
nslookup -type=A yourdomain.com
```

Make sure the old IP is no longer returned.

Also check for an unwanted AAAA record:

```
nslookup -type=AAAA yourdomain.com
```

Problem 6 — Firebase says "Minting certificate"

This means Firebase is creating the SSL certificate.

Usually, no action is required.

Wait for Firebase to finish the process.

Problem 7 — Firebase shows "Site Not Found"

Possible causes include:

1. The Hosting deployment hasn't happened.
2. The wrong Firebase project is being used.
3. The custom domain is still being configured.
4. SSL/custom-domain setup is not complete.

Check your deployment:

```
firebase hosting:channel:list --project YOUR-PROJECT-ID
```

A production channel called:

```
live
```

indicates that a live deployment exists.

35. Useful Commands

Check Firebase CLI

```
firebase --version
```

Login

```
firebase login
```

Login in headless environment

```
firebase login --no-localhost
```

Build React app

```
npm run build
```

Deploy

```
firebase deploy --only hosting
```

Deploy to a specific project

```
firebase deploy --only hosting --project YOUR-PROJECT-ID
```

List Hosting sites

```
firebase hosting:sites:list --project YOUR-PROJECT-ID
```

List Hosting channels

```
firebase hosting:channel:list --project YOUR-PROJECT-ID
```

Check DNS A record

```
nslookup -type=A yourdomain.com
```

Check DNS TXT record

```
nslookup -type=TXT yourdomain.com
```

Check IPv6 DNS

```
nslookup -type=AAAA yourdomain.com
```

36. Complete Deployment Checklist

Use this checklist when deploying another React/Vite website.

React application

- ☐ Website works locally
- ☐ `npm install` completed
- ☐ `npm run build` succeeds
- ☐ `dist/` exists
- ☐ Production build tested

Firebase

- ☐ Firebase project created
- ☐ Firebase Hosting enabled
- ☐ Firebase CLI installed
- ☐ Firebase CLI authenticated
- ☐ `.firebaserc` configured
- ☐ `firebase.json` configured
- ☐ Hosting deployed
- ☐ `.web.app` URL works

Custom domain

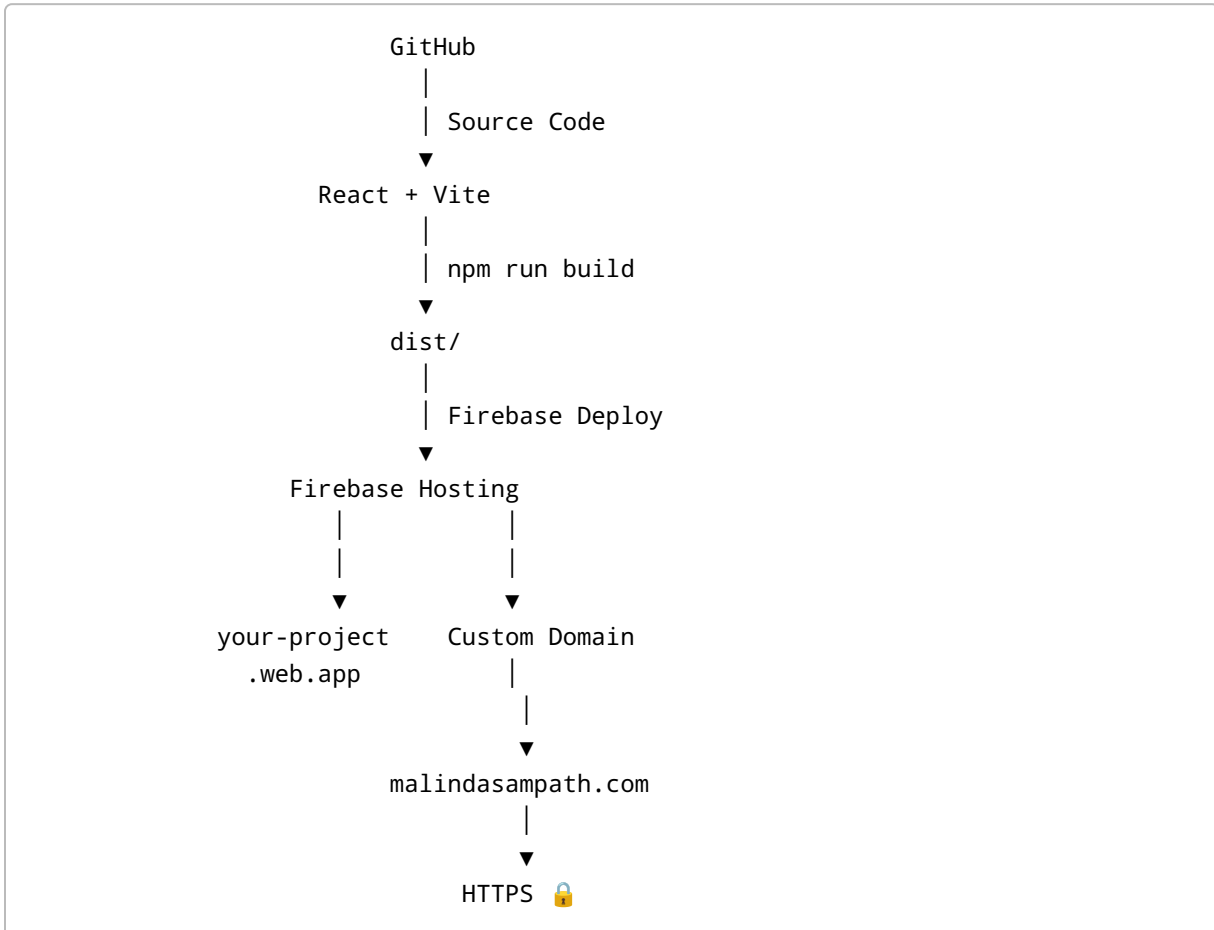
- ☐ Custom domain added in Firebase
- ☐ Firebase A record added to Hostinger
- ☐ Firebase TXT verification record added
- ☐ Old A record removed
- ☐ `www` CNAME configured
- ☐ DNS verified with `nslookup`
- ☐ Firebase domain ownership verified
- ☐ SSL certificate provisioned
- ☐ HTTPS works

Final test

- ☐ `https://yourdomain.com` works
 - ☐ `https://www.yourdomain.com` works
 - ☐ HTTPS lock appears
 - ☐ All images load
 - ☐ All routes work
 - ☐ Mobile layout works
-

37. Final Result

The completed setup is:



For this portfolio, the production deployment is therefore very simple:

```
npm run build
```

```
firebase deploy --only hosting --project YOUR-PROJECT-ID
```

The custom domain and DNS only need to be configured once. After that, future updates only require building and deploying the new version.